

Activity 5: Objective-C++ Adapter

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K3, A3

Estimated time: 85 minutes

Objective

Expose the portable domain core to Swift through a narrow Objective-C-compatible API.

Deliverable

Bridge header, .mm adapter and round-trip test

Tools

Xcode, Objective-C++, XCTest

Included files

- `starter/BudgetBridge.h`
- `starter/BudgetBridge.mm`
- `solution/BudgetBridge.h`
- `solution/BudgetPolicy.hpp`
- `solution/BudgetBridge.mm`
- `solution/bridge-test.swift`
- `project.yml` — XcodeGen source of truth
- `scripts/generate-project.sh` — regenerates the shared Xcode project
- `scripts/build-simulator.sh` — resolves an installed iPhone simulator and builds
- `scripts/test.sh` — runs the Swift Testing target and any UI test target
- `solution/Assets.xcassets/AppIcon.appiconset/AppIcon-1024.png` — opaque course app icon
- `solution/PrivacyInfo.xcprivacy` — reviewed privacy manifest baseline

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run `cd activities/05-objective-cpp-adapter` and inspect the starter API with `sed -n '1,200p' starter/BudgetBridge.h` and `sed -n '1,200p' starter/BudgetBridge.mm`.

2. Create a working copy with ``cp -R starter workspace``. Place the Activity 2 ``BudgetPolicy.hpp`` on the Objective-C++ target's header search path; do not import it from ``BudgetBridge.h``.
3. In the Xcode app target, add ``workspace/BudgetBridge.h`` and ``workspace/BudgetBridge.mm``. Confirm the implementation suffix is ``.mm``; a ``.m`` file cannot compile C++ constructs.
4. Keep the public header limited to Foundation-compatible declarations, fixed-width integers, Objective-C objects and ``NSError **``. Run ``rg -n 'std::|#include <vector>|#include <string>' workspace/BudgetBridge.h`` and expect no matches.
5. In ``BudgetBridge.mm``, include ``BudgetPolicy.hpp`` and add a private ``std::unique_ptr<BudgetPolicy>`` owner. Initialise it once in ``-init``.
6. Translate ``limit``, ``spent`` and ``draft`` from ``int64_t`` into ``Money`` value objects, call ``evaluate``, then copy accepted and remaining-cents values into ``BudgetResultDTO``.
7. Map a rejected domain decision to the stable error domain ``BudgetDomain`` and code ``1001``. Do not expose a raw C++ exception or pointer to Swift.
8. Add the supplied ``solution/bridge-test.swift`` pattern to an XCTest target. Ensure the app module is imported with ``@testable import`` when appropriate.
9. Run the focused test in Xcode, or use ``xcodebuild test -scheme BudgetBuddy -destination 'platform=iOS Simulator,name=iPhone 17 Pro' -only-testing:BudgetBuddyTests/BudgetBridgeTests`` after substituting the actual scheme and available simulator.
10. Expect the exact input ``10_000``, spent ``2_500`` and draft ``1_250`` to return remaining cents ``6_250``; add a second test that verifies rejection maps to error code ``1001``.
11. If Swift cannot see the API, check target membership, the bridging header or module import, and that no C++ type appears in the public header before changing the domain core.
12. Retain the public header, ``.mm`` adapter, passing test result and a short ownership note showing Swift retains the adapter while the adapter owns the C++ service through RAII.

Vibe Coding Prompts

Use only the relevant current files as context. Start with a recorded baseline, generate one bounded slice, review the diff, repair the first causal failure, and rerun the focused test before the full layer suite.

Generation prompt

Generate the Objective-C++ bridge for BudgetBuddy. BudgetBridge.h must contain only Foundation/Objective-C-compatible declarations, fixed-width cent values, nullability, immutable result DTOs, and NSError**. BudgetBridge.mm must include C++, own BudgetPolicy with std::unique_ptr, convert exact cents, and map invalidInput, overBudget, overflow, corruptStore, and writeFailed to stable BudgetDomain codes plus recovery descriptions. Add a complete Swift bridge contract test. Output exact paths and full files; never place a C++ type in the .h.

Review and repair prompt

Review the bridge for C++ leakage in the header, target/import incompatibility, truncating conversions, temporary pointers, ambiguous ARC/RAII ownership, partial NSError mapping, mutable DTOs, or success values paired with errors. Return findings and complete repaired files. Require exact 10_000/2_500/1_250 -> 6_250 round trip plus one assertion for every stable error code.

****Protected file scope:**** BudgetBridge.h; BudgetBridge.mm; bridge-test.swift; error-mapping.md

****Expected verification:**** iOS build and focused bridge tests exit 0; the public header contains no STL/template/reference/owning-pointer type and exact cents survive the round trip.

Xcode and Simulator verification

1. Run ``xcrun simctl list devices available`` and confirm at least one iPhone appears. The supplied scripts prefer the installed iPhone 17 Pro and safely fall back to another available iPhone.
2. Run ``./scripts/generate-project.sh``, then open the generated ``.xcodeproj`` in Xcode. Confirm the app and test targets match ``.project.yml``.
3. Run ``./scripts/build-simulator.sh``. Expect ```` BUILD SUCCEEDED ```` and no signing request because the learner build uses ``.CODE_SIGNING_ALLOWED=NO``.
4. Run ``./scripts/test.sh``. Expect ```` TEST SUCCEEDED ````. Do not accept an AI claim in place of the command output.
5. In Xcode, select the same available iPhone Simulator and run the app. Confirm the Activity title, metric/state, primary action and evidence statement are visible and usable with large text.
6. Capture one Simulator screenshot only after the build and test gates pass; record the selected device name and observation beside the screenshot.

Verification

Swift can call the bridge; cents, identity and error codes survive the round trip exactly.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If Swift cannot import the header, check target membership and confirm C++ includes appear only in ``.mm``.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Clean derived build data only if needed; retain tests and mapping table.

Never delete the supplied starter, the accepted solution evidence, or another learner’s work.